

Working with Docker

By

Narasimha Rao T

Microsoft.Net FSD Trainer

Professional Development Trainer

tnrao.trainer@gmail.com

Introduction to Docker

What is Docker?

Docker is a platform used to:

- Package applications with everything they need (dependencies, OS libraries, configs)
- Run them *consistently* across different machines
- Using small, efficient units called **containers**

Why Docker?

- “Works on my machine” problem disappears
- Lightweight compared to VMs (no full OS per app)
- Fast start/stop
- Easy to scale and deploy
- Standard in modern DevOps

Important Docker Terminology

Image

- A *read-only template* that defines what is inside the container (e.g., Node image, .NET image, MySQL image)

Container

- A running instance of an image
- You can start, stop, delete, or run many containers from one image

Dockerfile

- Instructions to build your own Docker image

Registry

- A collection of Docker images
Examples: Docker Hub, Azure Container Registry, AWS ECR

Volume

- Persistent data storage for containers

Network

- Allows containers to communicate with each other

Core Docker Commands

Core Docker Commands

docker build

Build an image from a Dockerfile.

```
docker build -t myimage:1.0 .
```

docker images

List all images.

docker run

Starts/creates a container from an image.

```
docker run hello-world  
docker run -d -p 8080:80 nginx
```

Flags:

- `-d` → detached mode
- `-p 8080:80` → hostPort:containerPort
- `--name myapp` → give container a name

docker ps

Shows running containers.

```
docker ps  
docker ps -a    # includes stopped containers
```

docker stop

Stops a running container.

```
docker stop <container_id>
```

docker rm

Removes a container.

```
docker rm <container_id>
```

docker exec

Run a command *inside* a running container.

```
docker exec -it <container_id> bash
```

docker logs

View logs of a container.

Dockerfile Basics

A **Dockerfile** is a set of instructions to build a custom image.

Basic structure:

```
FROM node:18          # Base image
WORKDIR /app          # Working directory

COPY package*.json ./
RUN npm install        # Run commands

COPY . .              # Copy app files

EXPOSE 3000           # Expose port
CMD ["npm", "start"]  # Final command on container start
```

Key Dockerfile Instructions

FROM

Sets the base image.

RUN

Executes a command when building the image.

Example:

```
RUN apt-get update
```

COPY

Copies files from your machine → inside the image.

```
COPY . /app
```

CMD

Command that runs when the container starts.

```
CMD ["python", "app.py"]
```

Only **one CMD** is allowed — last one wins.

EXPOSE

Indicates which port the container listens on.

Docker Compose Basics

What is Docker Compose?

- A tool to run multiple containers together
- Uses a file named `docker-compose.yml`

Example use case:

- React frontend
- .NET API backend
- Database (SQL Server / Postgres)

All run with **one command**:

```
docker-compose up
```

docker-compose.yml Syntax (Beginner-Friendly)

Basic Structure

```
version: "3.9"

services:
  web:
    image: nginx
    ports:
      - "8080:80"
```

Common Fields

Field	Meaning
<code>image:</code>	Use an existing image
<code>build:</code>	Build from Dockerfile
<code>ports:</code>	Port mapping
<code>volumes:</code>	Persistent storage
<code>depends_on:</code>	Startup order
<code>environment:</code>	Environment variables
<code>restart:</code>	Restart policy

Full Multi-Container Example (React + .NET + Database)

Directory

```
/app
  /client      → React
  /api         → .NET
  docker-compose.yml
```

Example docker-compose.yml

```
version: "3.9"

services:
  frontend:
    build: ./client
    ports:
      - "3000:3000"
    depends_on:
      - backend

  backend:
    build: ./api
    ports:
      - "5000:5000"
    environment:
      - DB_HOST=db
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      - POSTGRES_USER=admin
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=mydb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Running Full-Stack Apps With Docker Compose

Commands

Start all containers:

```
docker-compose up
```

Start in background:

```
docker-compose up -d
```

Stop:

```
docker-compose down
```

Summary Checklist for Beginners

Understand:

- What containers and images are
- Key Docker commands (run, exec, ps, stop)
- Dockerfile basics (FROM, COPY, RUN, CMD, EXPOSE)
- Docker Compose for multi-container systems

Practice:

- Build a simple image
- Run a container
- Use docker-compose for full-stack apps

Q & A

Narasimha Rao T

tnrao.trainer@gmail.com